Lab 7 Clarifications:

- **Use the RF24 library** with SPI wiring (CE/CSN + MISO/MOSI/SCK).
- **Power the radio at 3.3 V only** (never 5 V). Typical RX/TX current is ~12–13 mA; PA/LNA versions can need much more.
- **I/O pins on nRF24L01/+ are 5 V-tolerant** (per Nordic's spec), so you can connect Arduino Uno SPI pins directly (still power VCC at 3.3 V!).
- **Do not use a resistor divider for the radio's VCC.** Use a proper 3.3 V regulator (or the common "nRF24 adapter/base" board that includes one and decoupling).

Why a resistor divider is a bad idea for VCC

A divider produces a "voltage" that depends on load current. The nRF24's current **changes** (sleep ↔ RX/TX, and much higher for PA/LNA boards), so a divider will sag or ripple, risking brownouts and flaky links. A **low-impedance 3.3 V source** (LDO or DC-DC with local caps) is the right solution. The inexpensive **"nRF24L01 base/adapter" boards** you linked do exactly this: take 5 V input, regulate to 3.3 V, and provide bulk/decoupling caps sized for RF bursts. Keep using the Uno at 5 V logic; feed the radio 3.3 V from an adapter or a good LDO. The SPI lines can connect directly because Nordic lists the **inputs as 5 V tolerant**. (VCC must still be 3.3 V.)

Wiring (Uno ↔ nRF24L01+)

- **VCC → 3.3 V**, **GND → GND** (or to a 5 V-to-3.3 V adapter board's VCC input).
- **CE → D7** (any digital pin)
- **CSN (a.k.a. CS) → D8** (any digital pin)
- **SCK → D13**, **MOSI → D11**, **MISO → D12** (hardware SPI)
- **IRQ** (optional) → D2 (interrupt capable)

RF24 uses: RF24 radio(CE, CSN); and the hardware SPI pins under the hood.

Power tip: Add a **10 µF (or 47–100 µF for PA/LNA)** electrolytic near the radio's VCC+GND and a **0.1 µF** ceramic right at the header. This tames burst current and avoids "random no-ack" gremlins.

Transmitter (reads LDR, sends one byte)

```
#include <SPI.h>
#include <RF24.h>
#include <LiquidCrystal.h>

LiquidCrystal lcd(12,11,5,4,3,2);   // keep your LCD wiring
RF24 radio(7, 8);                   // CE=D7, CSN=D8

const byte pipeAddr[6] = "L7TX1";
const int  LDR_SENSOR_PIN = A0;

void setup() {
  lcd.begin(20,4);
  lcd.setCursor(0,0); lcd.print("LDR BASED LIGHT");
  lcd.setCursor(0,1); lcd.print("Intensity control");
  lcd.setCursor(0,2); lcd.print("Transmitter");

  radio.begin();
  // Lower data rate = more robust
  radio.setDataRate(RF24_250KBPS);
  radio.setRetries(5, 15);          // delay,count
  radio.openWritingPipe(pipeAddr);
  radio.stopListening();             // TX mode
}

void loop() {
  int raw = analogRead(LDR_SENSOR_PIN); // 0..1023
  uint8_t level = raw / 100;            // ~0..10
  lcd.setCursor(0,3); lcd.print("LDR LEVEL:      );
  lcd.setCursor(11,3); lcd.print(level);

  radio.write(&level, sizeof(level));   // send one byte
  delay(100);
}
```

Receiver

```
#include <SPI.h>
#include <RF24.h>
#include <LiquidCrystal.h>

LiquidCrystal lcd(13,12,11,10,9,8);     // keep your LCD wiring
RF24 radio(7, 8);                       // CE=D7, CSN=D8

// Dimmer control lines (as before)
#define DIMMER_PIN_A 7
#define DIMMER_PIN_B 6
#define DIMMER_PIN_C 5
#define DIMMER_PIN_D 4

void setDimmer(bool A,bool B,bool C,bool D){
  digitalWrite(DIMMER_PIN_A, A);
  digitalWrite(DIMMER_PIN_B, B);
  digitalWrite(DIMMER_PIN_C, C);
  digitalWrite(DIMMER_PIN_D, D);
}

const byte pipeAddr[6] = "L7TX1";

void setup() {
  lcd.begin(20,4);
  lcd.setCursor(0,0); lcd.print("LDR BASED LIGHT");
  lcd.setCursor(0,1); lcd.print("Intensity control");
  lcd.setCursor(0,2); lcd.print("Receiver");

  pinMode(DIMMER_PIN_A, OUTPUT);
  pinMode(DIMMER_PIN_B, OUTPUT);
  pinMode(DIMMER_PIN_C, OUTPUT);
  pinMode(DIMMER_PIN_D, OUTPUT);
  setDimmer(0,0,0,0);

  radio.begin();
  radio.setDataRate(RF24_250KBPS);
  radio.openReadingPipe(1, pipeAddr);
  radio.startListening();
}

void loop() {
  if (radio.available()) {
    uint8_t level;
    radio.read(&level, sizeof(level));

    lcd.setCursor(0,3); lcd.print("LDR LEVEL:      );
    lcd.setCursor(11,3); lcd.print(level);

    if (level <= 2)      setDimmer(0,0,0,1);
    else if (level <= 5) setDimmer(0,0,1,0);
    else if (level <= 8) setDimmer(0,1,0,0);
    else                 setDimmer(1,0,0,0);
  }
}
```

This preserves our Lab 7 behavior (one-byte “intensity” payload with LCD + dicrete dimmer steps), but now uses the **RF24** stack properly over SPI.

Extra notes on “level shifting”

- You **don’t** need level shifters for Uno→nRF24L01+ SPI if you’re using a genuine/compatible **nRF24L01 or nRF24L01+** (Nordic lists **5 V-tolerant inputs**). Still, always power VCC at 3.3 V.
- If you ever use a **different 3.3 V radio** that is *not* 5 V-tolerant, use a proper MOSFET-based **logic-level converter** on MOSI/SCK/CSN/CE (MISO usually reads fine at 3.3 V on a 5 V AVR, since VIH ≈ 0.6·Vcc ≈ 3.0 V).
- Avoid **resistor dividers on SPI signals** at higher clocks; they form RCs and distort edges. If you must (slow clocks), keep R’s low and wiring short—but a proper level shifter is better practice. (Community guidance; vendor-agnostic.)